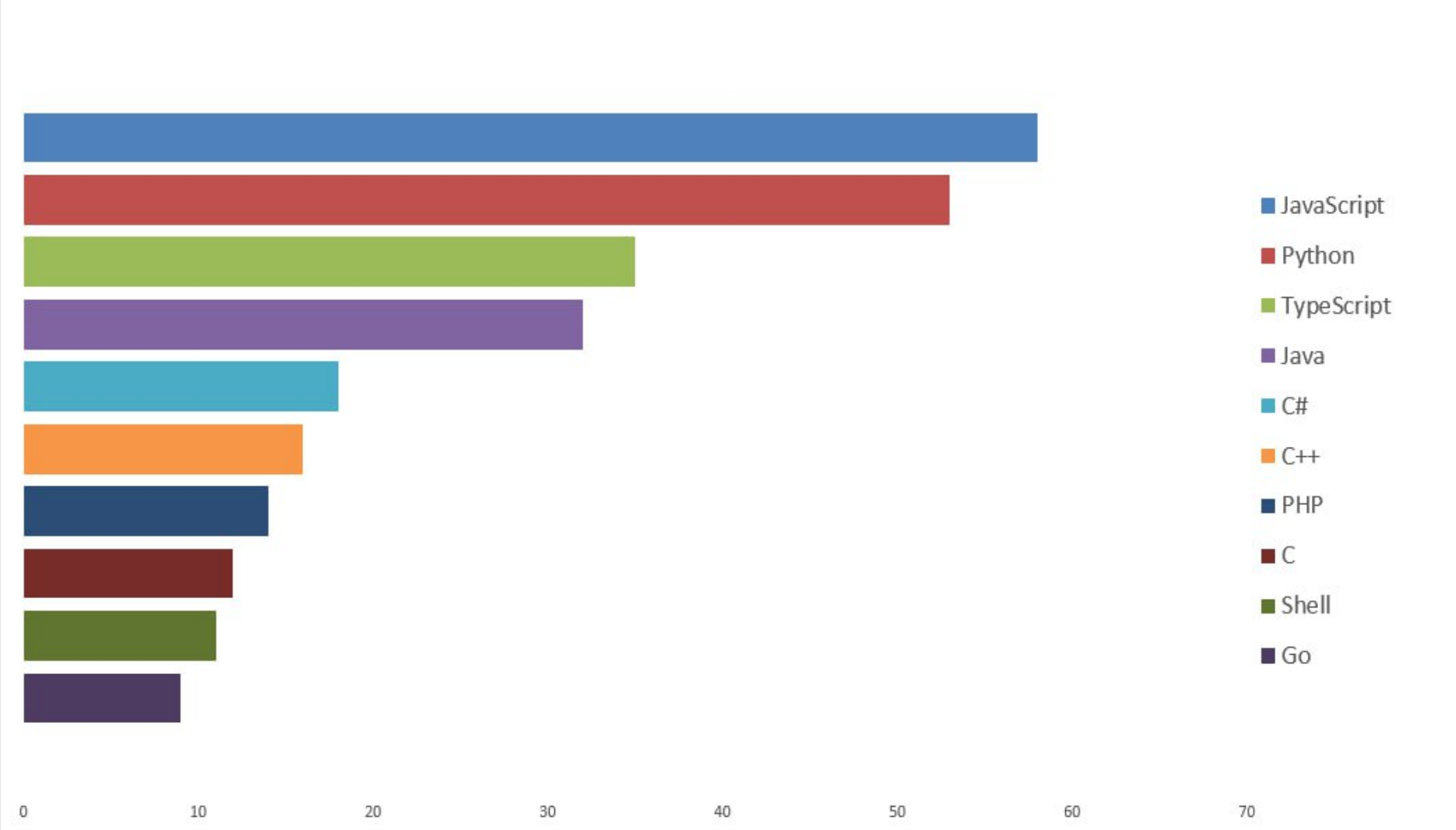
Часть 2. Программирование на языке C#

МГТУ им. Н.Э. Баумана
Факультет Информатика и системы управления
Кафедра Компьютерные системы и сети
Дисциплина Алгоритмизация и программирование
Лектор: д.т.н., проф. Иванова Галина Сергеевна

Что учить в 2024 году? Рейтинг GitHub



Введение. Платформа .NET

Платформа – *среда времени выполнения программы*, включающая программные, программно-аппаратные и/или аппаратные компоненты, обеспечивающие функционирование программ. Это может быть операционная система, браузер или библиотеки времени выполнения и т.п.

Платформа .NET кроме этого включает средства разработки программ на нескольких языках программирования, а также механизмы поддержки баз данных, электронной почты и т.п.

Платформа .NET обеспечивает для программного обеспечения:

- *кроссплатформенность* – возможность работы под разными ОС;
- *безопасность* – исключение несанкционированных действий;
- *надежность* – способность функционировать в определенных условиях;
- *наличие* больших библиотек готовых компонентов;
- межъязыковое взаимодействие – возможность использования для разработки нескольких языков программирования.

Версии .NET

2002 г. – первая версия - .NET Framework - только для Windows;

2016 г. - .NET Core – кроссплатформенная открытая среда для Windows, Mac OS и Linux.

Сейчас

.NET Framework 4.8.1

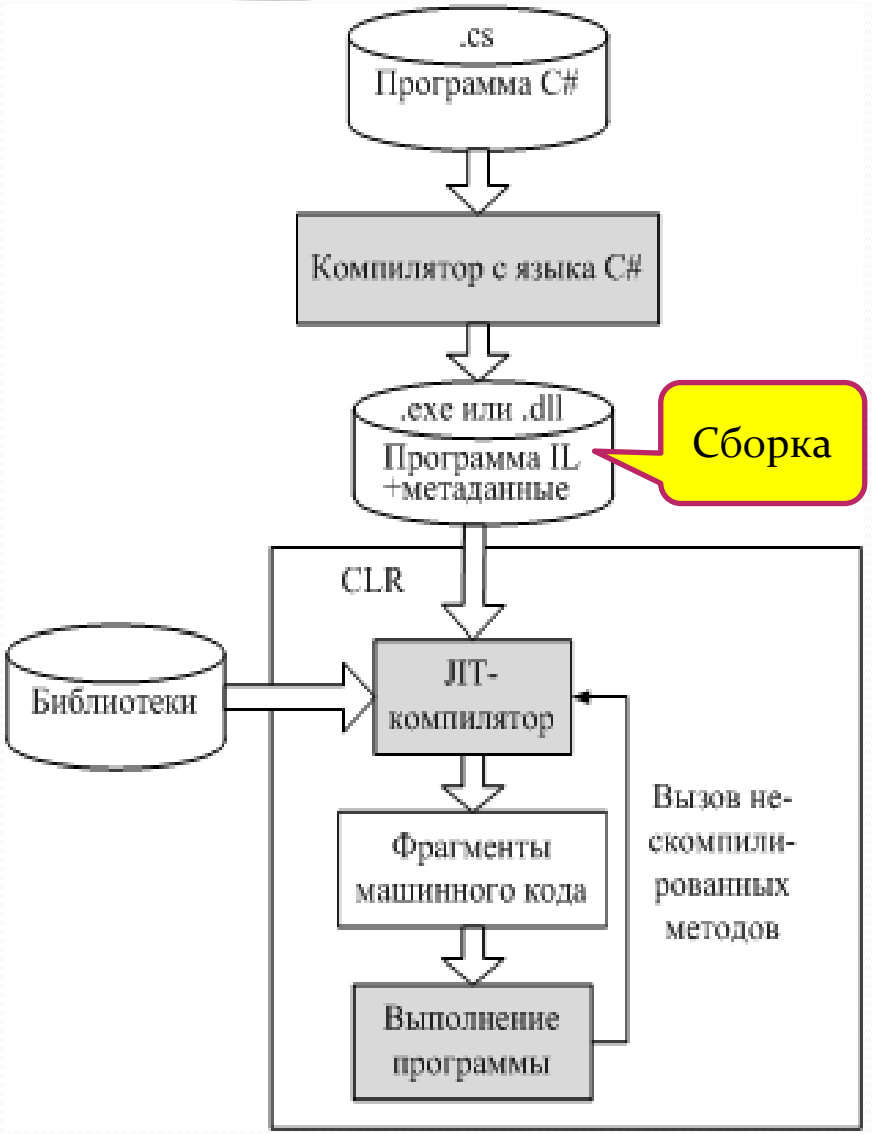
.NET 8 - 9

Процесс выполнения программы в .NET

Для обеспечения переносимости компиляторы переводят программу в промежуточный язык **Microsoft Intermediate Language**, (MSIL, IL), который не содержит команд, зависящих от языка, операционной системы и типа компьютера.

Программа на IL выполняется в общеязыковой среде выполнения **Common Language Runtime** (CLR), которая может быть реализована для любой ОС. Управляет памятью, потоками, выполнением кода, проверкой безопасности, компиляцией и др. системными службами.

Частью CLR является специальный компилятор **Just In Time**(JIT), переводящий программу с языка IL в машинные команды соответствующего процессора. Причем переводятся только конкретно выполняемые в настоящий момент части программы и сохраняются для дальнейшего использования.



Сборка

Сборка - это файл, который содержит программу или библиотеку кода, элементы управления, ресурсы или другую информацию, которая может быть использованы программой. Сборка создается как единый проект в Visual Studio или аналогичной среде разработки и компилируется в исполняемый файл (EXE) или динамическую связанную библиотеку (DLL). Сборки хранят четыре типа информации:

- манифест сборки (имя сборки, номер версии, локализация, подпись, список файлов, сведения о ссылках и т.п.);
- метаданные – сведения о данных для многоязыковых приложений;
- код на промежуточном языке Microsoft (MSIL);
- ресурсы – изображения, значки, звуки, локализованный текст.

Места нахождения сборок:

1) глобальный кэш сборок (GAC), который используется для общих сборок;

2) папки приложения хранят частные сборки;

3) в сети.

Глава 1.
Основы языка C#

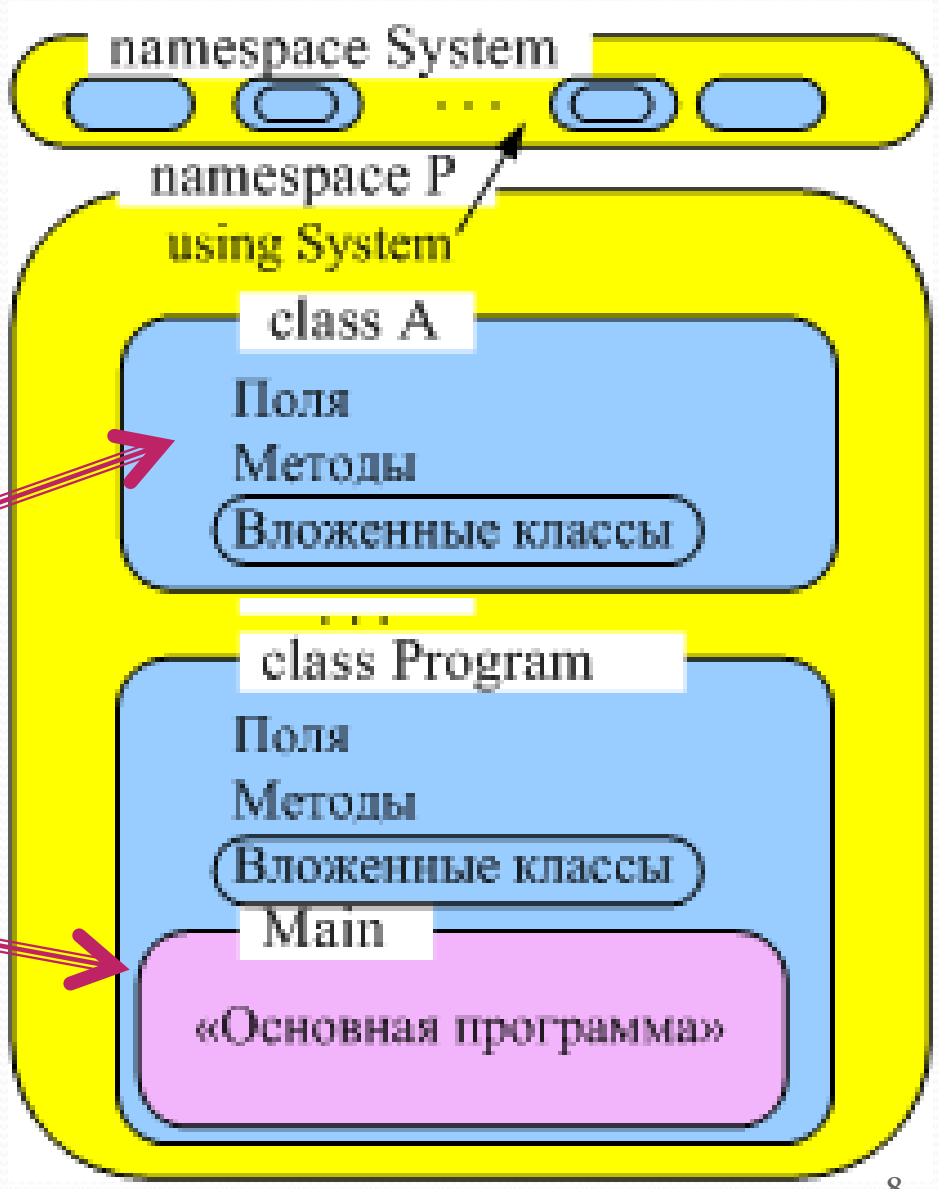
МГТУ им. Н.Э. Баумана
Факультет Информатика и системы управления
Кафедра Компьютерные системы и сети
Дисциплина Алгоритмизация и программирование
Лектор: д.т.н., проф. Иванова Галина Сергеевна

1.1 Структура программы на C#

Программа использует несколько *пространств имен*, которые включают описания классов, в том числе стандартных, необходимых программе.

Классы могут содержать поля, методы и *вложенные классы*.

Программа должна включать класс, содержащий *метод с именем* Main. С этого метода начинается выполнение программы.

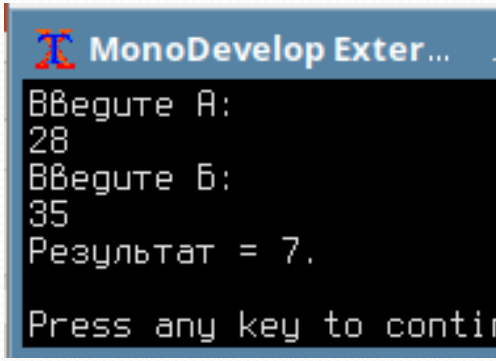


Пример консольной программы

```
using System;

namespace Ex01_01
{
    class Program
    {
        public static void Main (string[] args)
        {
            int A,B;
            Console.WriteLine("Введите А:");
            A = Convert.ToInt32(Console.ReadLine());
            Console.WriteLine("Введите Б:");
            B = Convert.ToInt32(Console.ReadLine());
            while (A != B)
            {
                if (A > B)
                    A = A - B;
                else B = B - A;
            }
            Console.WriteLine("Результат = {0}.",A);
        }
    }
}
```

Program
Main()



1.2 Синтаксис языка

Алфавит языка C#:

- буквы: A..Z, a..z, символ "_" и А..Я, а..я - все в кодировке Unicode (UTF-16);
- цифры: 0..9, A..F;
- специальные символы: +, -, *, /;
- пробелы и символы табуляции.

Из символов строятся лексемы.

Лексема – минимальная единица языка, имеющая самостоятельный смысл:

- идентификатор, например: **`a`**, **`МАРИЯ`**, **`F_21`**, **`_f`**;
- служебный символ (слово), например: **`struct`**, **`+=`**;
- разделитель, например: **`{ , .`**;
- литерал, например: **`null`**, **`85`**, **`764U`**.

Компилятор различает строчные и прописные буквы

Рекомендуемые нотации записи идентификаторов:

- *нотация Pascal* – каждое слово начинается с прописной буквы:

  `MaxLength`, **`MyFatCat`**;
- *нотация Camel* – с прописной буквы начинаются все слова, кроме первого: **`maxLength`**, **`myFatCat`**.

1.3 Типы данных

Все типы данных .NET наследуются от класса **`Object`** и делятся на две группы:

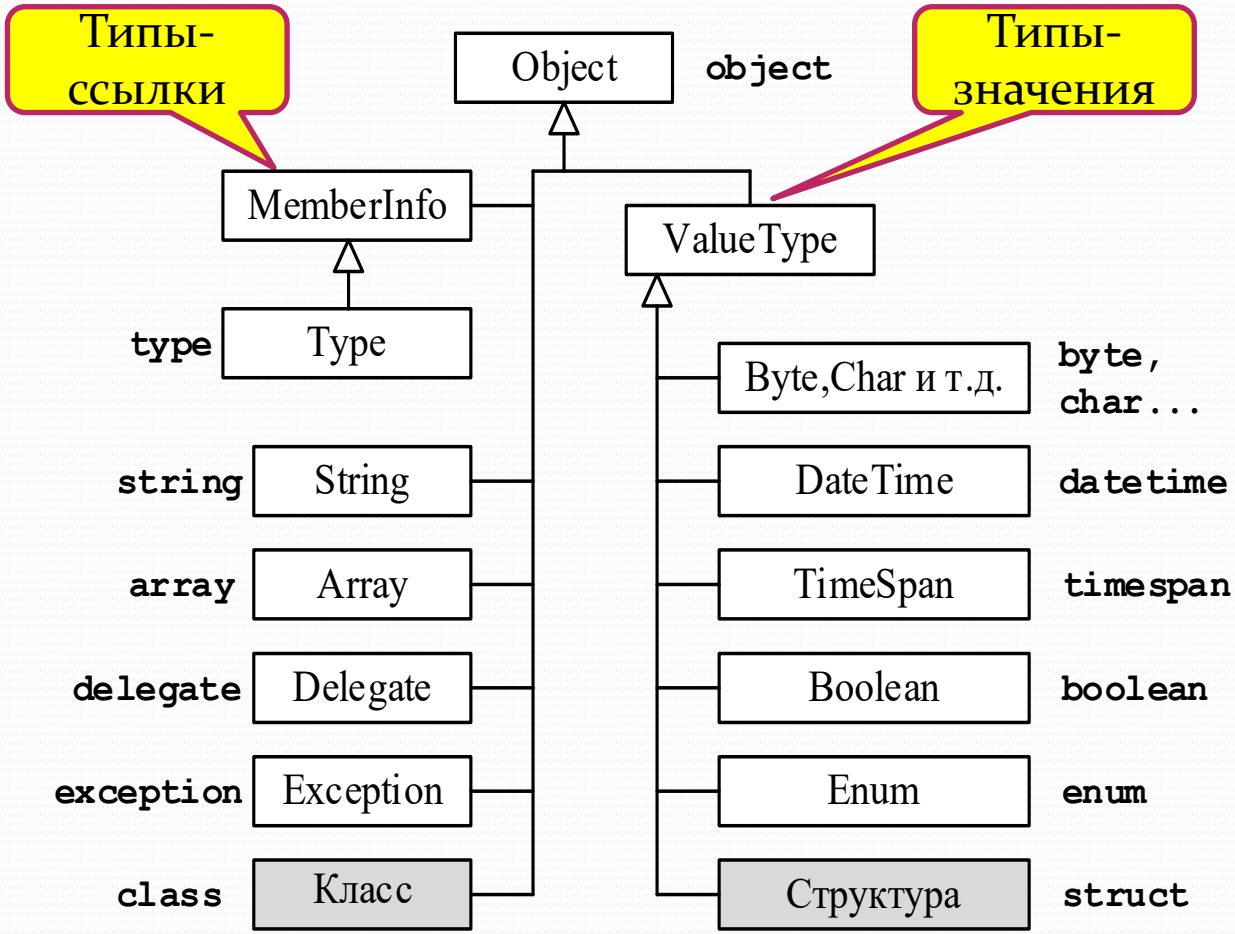
- *типы-значения*;
- *типы-ссылки*.

Данные типа-значения и адреса данных типа-ссылки *обычно* размещаются в стеке.

Значения, на которые указывают ссылки, размещаются в динамической памяти (куче).

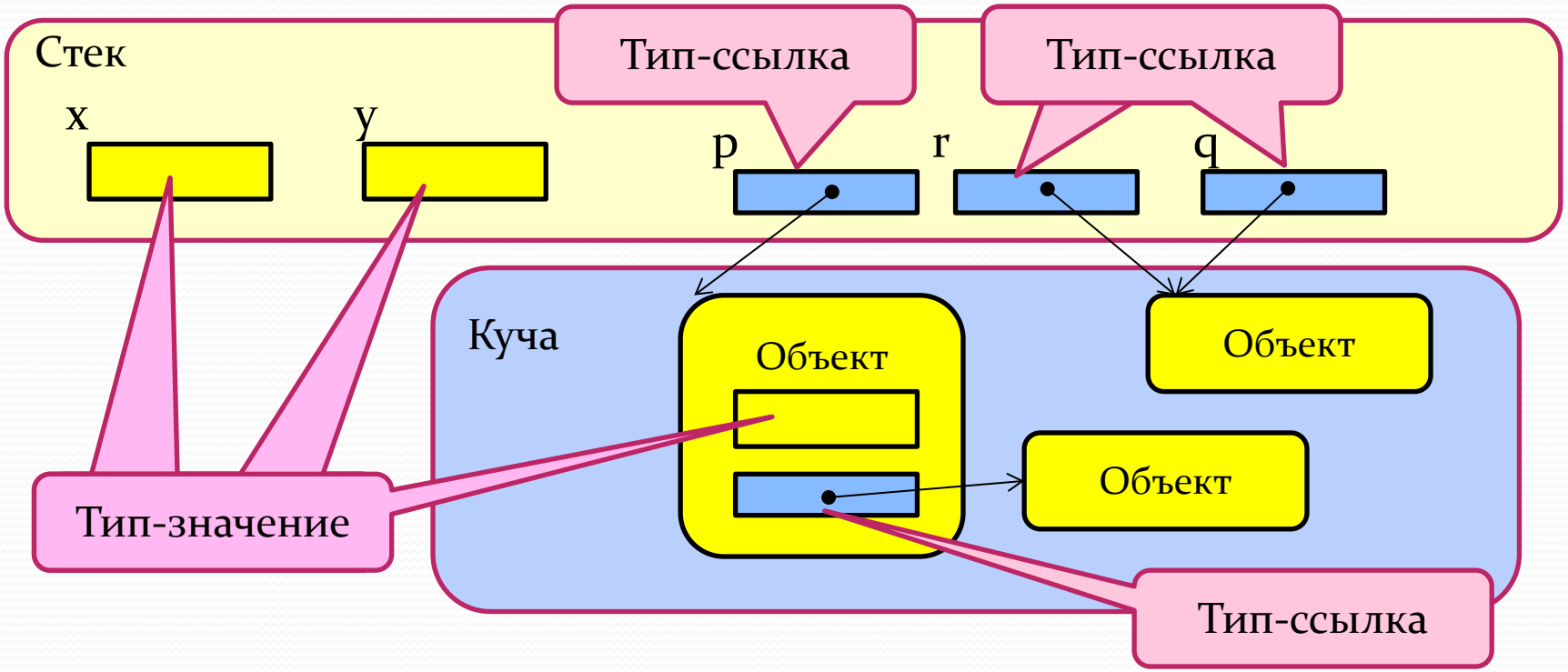
За освобождением памяти в куче следит *сборщик мусора*.

Фрагмент иерархии классов данных .NET и соответствующие типы данных C#



Различие типов-значений и типов-ссылок

Внимание! Не все поля типа-значения находятся в стеке! Если поле типа-значения, например, входит в объект, то оно располагается в куче…




```
x == y
```

Равны значения

```
q == r
```

Равны адреса!!!

Встроенные типы (структуры и классы) данных

Тип (Класс)	Служебное слово C#	Тип .NET	Диапазон	Размер, байтов
Логический	bool	Boolean	true, false	1
Целые	sbyte byte short ushort int uint long ulong	Sbyte Byte Int16 UInt16 Int32 UInt32 Int64 Uint64	-128..127 0..256 -32768..32767 0..65535 $-2*10^9..2*10^9$ $0..4*10^9$ $-9*10^{18}..9*10^{18}$ $0..18*10^{18}$	1 1 2 2 4 4 8 8
Вещественный*	float double	Single Double	$1.5*10^{-45}..3.4*10^{38}$ $5.0*10^{-324}..1.7*10^{308}$	4 8
Финансовый	decimal	Decimal	$1.0*10^{-28}..7.9*10^{28}$	16
Символьная	char	Char	0x0000..0xffff	2
Строковый	string	String		
Базовый тип	object	Object	может хранить данные любого типа	4 или 8

* Для вещественных чисел диапазон указан по абсолютной величине.

Литералы

Тип константы	Описание	Примеры
Логическая	`true` или `false`	`true`
Целая десятичная Целая шестнадца-теричная	[Знак] Цифра {Цифра} [Суффикс] {0x|0X} Цифра$_{16}$ {Цифра$_{16}$} [Суффикс]	-15 -15L 38UL 0xA 0X2FU
Вещественная с фикс. точкой Вещественная с плав. точкой	[[Знак] Цифры].Цифры[Суффикс] [[Знак]Цифры].Цифры {e|E} [Знак]Цифры[Суффикс]	+5.2 .001d -5.2F 0.1e5D
Символьная	'Символ'	'A' '\0' '\u00F2'
Строковая	"Символ {Символ}"	"\tЭто пример"
Адресная	-	null

Суффикс уточняет тип: U,u – unsigned; L,l – long; F,f – float; D,d – double; M,m – decimal. По умолчанию целые – int, вещественные – double, если умещаются, иначе – минимальный до длине достаточный тип.

Класс System.Object

Класс System.Object – корень иерархии типов - включает следующие методы:

Компонент	Тип	Описание
Epsilon	Поле - константа	для типов Float и Double – минимальное значение переменной, такое что **1.0+ Epsilon != 1.0**;
Equals	Метод	Возвращает true, если вызывающий объект и параметр ссылаются на одну область памяти, например: **bool b = a.Equals(w);**
Equals	Статический метод	Возвращает true, если оба параметра-ссылки ссылаются на одну область памяти или для типов-значений равны и типы, и значения, например: **bool b = Object.Equals(w,v);**
GetHashCode	Метод	Возвращает целочисленный хеш-код объекта – число, однозначно идентифицирующее объект
GetType	Метод	Возвращает тип полиморфного объекта типа Type, например: **if (obj.GetType()!=this.GetType()) ...;**
ToString	Метод	Возвращает: для типов-значений – представление значения в виде строки, для ссылочных – название класса, например: **string s = obj.ToString();**

1.4 Переменные и поименованные константы

Переменные можно объявлять:

- внутри класса – ***поля*** – при объявлении по умолчанию инициализируются нулевыми значениями соответствующего типа;
- внутри метода - ***локальные переменные методов*** – автоматически не обнуляются и должны всегда инициализироваться в программе перед использованием, видимы внутри блоков, где объявлены.

Синтаксис объявления *простых* переменных типов-значений:

Тип Имя [= **Выражение**] {, **Имя** [= **Выражение**] };

Примеры:

```
int x, y = 5;
float fn = 2.5f;
```

Суффикс указан, чтобы сразу создать литерал типа float, так как по умолчанию будет создан литерал типа double, который в переменную float не умещается.

Неявная типизация данных

Синтаксис:

`var` Имя = Выражение{, Имя = Выражение};

Компилятор *выводит* тип, исходя из типа инициализирующего значения.

Примеры:

1) `var hello = "Строка";` // тип string

2) `var c = 20;` // тип int

3) `var c;` // ошибка: тип не определен (не из чего выводить)

`c = 20;`

4) `var c = null;` // ошибка: тип не определен (нельзя использовать null)

Для уточнения типа можно использовать метод GetType() класса Object :

`hello.GetType()` // вернет System.String

Область действия переменной

Область действия переменной начинается в точке ее объявления и заканчивается при завершении блока (), в котором она объявлена.

В области действия имя переменной должно быть *уникальным*.

Однако, следует иметь в виду:

```
class A
{
    int x = 10;        // поле объекта this.x
    void one()
    {
        int x,y;  // локальные переменные метода
        {
            int x;    // Ошибка!! x – не уникальна!
        }
        y = this.x;
        …
    }
```

Именованные константы

Объявление именованных констант:

`const` Тип Имя = Выражение{, Имя = Выражение};

Пример:

```
const int flag = 1;
```

Выражение, инициализирующее константу, должно быть *вычисляемым* на этапе компиляции.

Создание новых полей

Операция **new** выполняет:

а) выделение памяти;

б) инициализацию поля данных.

Может применяться как к ссылочным типам, так и типам-значениям, но для типов-значений обычно не применяется.

Синтаксис:

Тип Имя_объекта = new Тип ([Аргументы]);

Примеры:

```
Dog Reks = new Dog("Рекс");

int j = new int();    // то же, что и int j = 0;
```

Перечисления

Перечисление – тип, все возможные значения которого задаются последовательностью поименованных констант.

Синтаксис:

[**Атрибуты**] **Спецификаторы** **enum** **Имя** [**: Базовый тип**] **Тело;**

Спецификаторы: **new, public, protected, internal, private**.

Базовый тип по умолчанию – **int**, однако может быть любым целочисленным, кроме **char**.

Примеры:

1) `enum Радуга {Красный,Оранжевый,Желтый,Зеленый,Синий, Фиолетовый};`

2) `enum Nums {two = 2, three, four, ten = 10, eleven, fifty = ten + 40 };`

Выражения должны быть вычисляемыми на этапе компиляции

3)
```
enum DayTime:byte {Morning,Afternoon,Evening,Night}
DayTime dayTime = DayTime.Morning;
if(dayTime == DayTime.Morning)…
```

1.5 Выражения

Выражение - правило вычисления значения. Состоит из операндов (переменные, литералы и поименованные константы) и операций:

Категория (по приоритетам)	Знаки операций
Первичные (имеют самый высокий приоритет)	`. () [] x++ x-- new typeof checked unchecked`
Унарные	`+ - ! ~`(поразрядное НЕ) `++x --x (<Тип>)`
Мультипликативные	`* / %`(применимо и к вещественным числам)
Аддитивные	`+ -`
Сдвига	`<< >>`
Отношения и проверка типа	`< > <= >= is as`
Проверка на равенство	`== !=`
Поразрядные логические	`&` `^`(исключающее ИЛИ) `|`
Условные логические	`&& ||`
Условная операция	`?:`
Присваивания	`= *= /= =% += -= <<= >>= &= ^= |=`

Без проверки на переполнение

Преобразование типов при выполнении операций

Если операнды арифметических операций разных типов, то

неявно выполняются преобразования, *исключающие потерю значимости*:

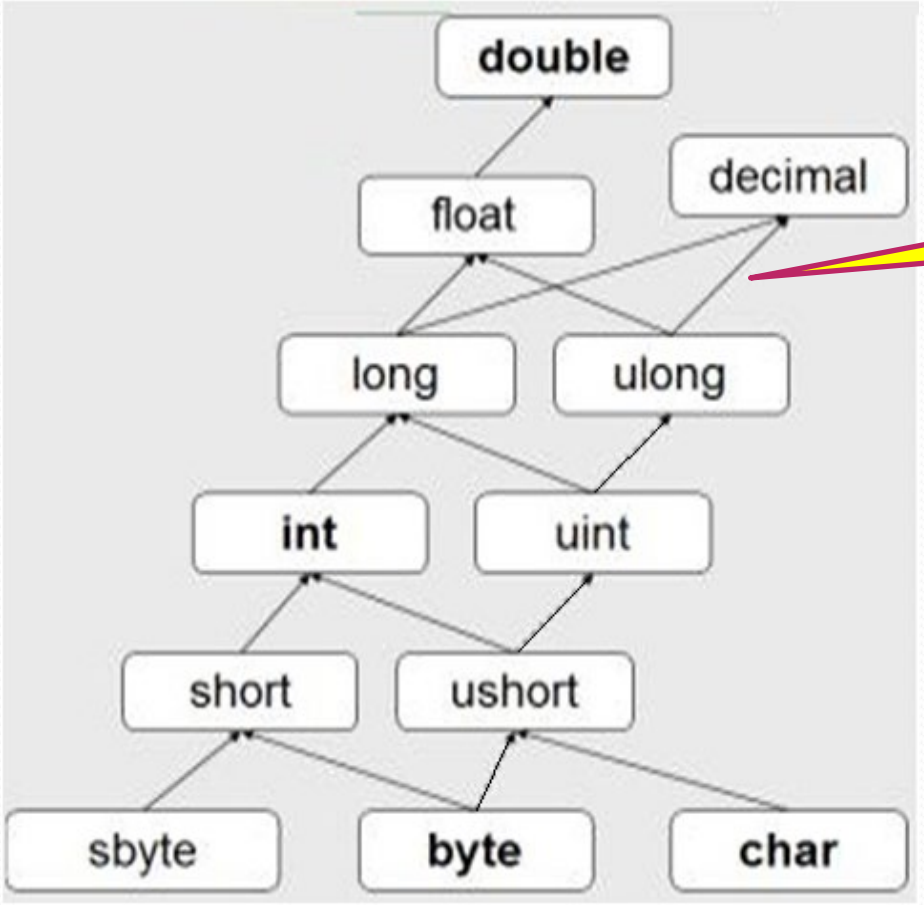
- при выполнении операции над целыми и вещественными данными разных типов - к более длинным типам;
- при выполнении операции над короткими целыми (менее 4 байт) – к типу int;
- при выполнении операции над целым и вещественным числами – целых к вещественным:

```
char -> int -> long -> float -> double
```

Преобразования, которые *могут привести к потере значимости*, например, вещественных в целые и финансовых в вещественные, а также длинного числа в короткое, требуют ***явного*** переопределения типа.

Неявное приведение типов

Неявное приведение возможно для следующих стандартных типов:



Неявное преобразование возможно

```
byte a; short b; => a*b          // результат типа int
float f1; double d1; => f1*d1    // результат типа double
                     => a+f1     // результат  типа float
```

Явное преобразования типов

При невозможности неявного преобразования необходимо выполнять явное преобразование числовых типов:

- с помощью операции **(Тип) Имя_переменной>** , например:

1) `int k = 100;  char c = (char)k;` // 'd'
2) `long b = 300;  int a = (int) b;` // данные не теряются
3) `byte d = (byte) a;` //??? , данные теряются
4) `decimal originalValue = 25.5M;`
   `int casted = (int)originalValue;` // результат 25

- соответствующим методом класса **Convert**, например:

1) `int i = Convert.ToInt32("123");`
2) `bool b = true;`
   `double d = Convert.ToDouble(b);`
3) `decimal originalValue = 25.5M;`
   `int converted = Convert.ToInt32(originalValue);` // 26

если преобразование невозможно, то метод генерирует исключение FormatException.

1.6 Стандартные математические функции

Определены в классе System.Math.

Статические поля:

- значение π: **`PI`**;
- значение e: **`E`**.

Основные статические методы:

- тригонометрические: **`Sin(x)`**, **`Cos(X)`**, **`Tan(x)`** (углы - в радианах);
- обратные тригонометрические: **`ASin(x)`**, **`ACos(x)`**, **`ATan(x)`**, **`ATan2(y/x)`**;
- гиперболические: **`Tanh(x)`**, **`Sinh(x)`**, **`Cosh(x)`**;
- экспонента и логарифмы: **`Exp(x)`**, **`Log(x)`**, **`Log10(x)`**;
- абсолютное значение, корень и знак: **`Abs(x)`**, **`Sqrt(x)`**, **`Sign(x)`**;
- округление: **`Round(x)`**, **`Celling(x)`** (округление до большего), **`Floor(x)`** (округление до меньшего);
- минимум и максимум: **`Min(x,y)`**, **`Max(x,y)`**;
- степень и остаток: **`Pow(x)`**, **`IEEReminder(x,y)`**;

1.7 Вывод консольного приложения

Для выполнения операций вывода используют статические методы класса System.Console:

- **`Console.Write(Выводимая_строка)`** – вывод строки без перехода на следующую;
- **`Console.WriteLine(Выводимая_строка)`** – вывод строки с переходом на следующую.

Методы перегружены для всех стандартных типов данных и форматного вывода.

Примеры с параметрами-строками

```
int i = 25; double Res = 27.1e-1;
Console.Write("Результат = "); // строка
Console.WriteLine(Res);        // вещественная переменная
Console.WriteLine("R = " + Res.ToString());
                               // объединение строк
Console.WriteLine("R = " + Res.ToString("C"));
    // объединение строк с добавлением национальной валюты
Console.WriteLine("R = " + Res);   // с неявным
                                   // преобразованием числа в строку
```

```
Результат = 2,71
R = 2,71
R = 2,71 руб.
R = 2,71
```

Форматирование строк (перегруженный метод)

Синтаксис вызова метода:

`WriteLine`("Строка,_содержащая_форматы", Список_аргументов);

Синтаксис формата в строке:

{**Номер_аргумента**[,**Мин.ширина_поля**[

(**:Спецификатор_формата**[**Число**] | **:Шаблон**)]]};

Спецификаторы формата:

C (или c) – Currency – форматировать как национальную валюту;

F (или f) – форматировать как число с фиксированной точкой;

G (или g) – форматировать в общем виде;

P (или p) – форматировать как процент;

X (или x) – форматировать как шестнадцатеричное значение.

Число – количество цифр в дробной части выводимого вещественного числа.

Шаблоны форматирования

Шаблон определяет вид выводимого значения:

- означает вставку значащей цифры числа;

0 (ноль) – означает вставку значащей цифры или незначащего нуля;

'Текст' – выводимый текст.

Примеры:

Число	Шаблон	Вид
1,12345	00.00	01,12
1,12345	#.##	1,12
0,1	00.00	00,10
0,1	#.##	,1

Примеры вывода форматированных строк

```
int i = 25;                    double Res = 27.1e-1;
Console.WriteLine("Res = {0} i = {1}", Res, i);
Console.WriteLine("Res = {0,6:C} i = {1,2:X}", Res, i);
Console.Write("Res = {0}\nRes = {1:F2} i = {1:G3}",
                                                Res, Res, i);
Console.WriteLine("Res = {1,4:#.##} i={0,3:#.###}",
                                                 Res,i);
Console.WriteLine("Res = {0,6:0.# ' руб '}", Res);
```

```
Res = 2,71 i = 25
Res = 2,71 руб. i = 19
Res = 2,71
Res = 2,71 i = 2,71Res =   25 i=2,71
Res = 2,7  руб
```

Интерполяция строк

Интерполированная строка содержит интерполяционные выражения.

Синтаксис подстроки с выражением интерполяции:

{ **Выражение** [, **Длина**][: **Формат**]}

Длина – константное выражение – минимальное количество символов подстроки. Если число положительное, то формируемая подстрока выровнена по правому краю, отрицательное – по левому.

Формат – строка форматирования.

$ – признак интерполированной строки

Примеры:

```
Console.WriteLine($"Результат = {Res}");
Console.WriteLine($"|{"Left",-7}|{"Right",7}|");
Console.WriteLine($"{Math.PI,16};
Console.WriteLine($"{Math.PI,15:F3};
```

F – формат с фикс. точкой,3 цифры после точки

```
Результат = 2,71
|Left   |  Right|
3,14159265358979
          3,142
```

1.8 Ввод консольного приложения

Для выполнения операций ввода используют статические методы класса System.Console:

- **`ReadLine()`** – ввод строки, возвращает строку типа string? или null, вводит данные, включая код Enter;
- **`Read()`** – возвращает значение типа int – код символа или -1, если значащих символов во входном потоке нет. Код Enter метод Read() не вводит!!!
- **`ReadKey()`** – возвращает код символа или функциональной клавиши.

Примеры:

1) `string s? = Console.ReadLine();` // кодировка UTF-16!

2) `char ch = (char)Console.Read();` // кодировка UTF-16!

3) `Console.ReadLine();` // ввод Enter

Пример Console.ReadKey()

```
using System;
class Example
{
   public static void Main()    {
      ConsoleKeyInfo cki;  // объект для записи информации о клавише
      Console.WriteLine("Press combination of CTL, ALT, and SHIFT, and a console key.");
      Console.WriteLine("Press the Escape (Esc) key to quit: \n");
      do       {
         cki = Console.ReadKey(true);
         Console.Write("You pressed ");
         if ((cki.Modifiers & ConsoleModifiers.Alt) != 0) Console.Write("ALT+");
         if ((cki.Modifiers & ConsoleModifiers.Shift) != 0) Console.Write("SHIFT+");
         if ((cki.Modifiers & ConsoleModifiers.Control) != 0) Console.Write("CTL+");
         Console.WriteLine("{0} (character '{1}')", cki.Key, cki.KeyChar);
      } while (cki.Key != ConsoleKey.Escape);
   }
}
```

Результат примера

```
C:\Users\Galina\source\repos\ConsoleApp2\bin\Debug\net8.0\Cons...
Press any combination of CTL, ALT, and SHIFT, and a console key.
Press the Escape (Esc) key to quit:

You pressed CTL+F (character '')
You pressed ALT+H (character 'h')
You pressed ALT+L (character 'l')
You pressed J (character 'J')
```

Преобразование строки в число

Преобразование строки в число осуществляют:

- с помощью методов класса **`Convert`**, при невозможности преобразования формирует исключение **`FormatExeption`**;
- с помощью метода **`Parse`**, который есть в каждом числовом классе, при невозможности преобразования формирует исключение **`FormatExeption`**;
- с помощью метода **`TryParse`** также числового класса, при невозможности преобразования возвращает false.

Примеры:

1) `int ik = Convert.ToInt32("123");`

2) `int i = int.Parse("123");`

`double d = double.Parse("123,45");`

3) `int ir;`

`bool result = int.TryParse("123", out ir);`

Разделитель определяется настройкой операционной системы

Обеспечение независимости разделителя от настройки операционной системы

```
using System.Globalization;

IFormatProvider formatter = new NumberFormatInfo
{ NumberDecimalSeparator = "." };


double number = double.Parse("23.56", formatter);
Console.WriteLine(number);    // 23,56
```

Объявленный разделитель

1.9 Простейшая линейная программа

Ex1.02

```
using System;
namespace Ex1_02
{
  class Program
  {
      public static void Main(string[] args)
      {
            int a,b,c;
            string s;
            Console.WriteLine("Введите a и b");
            s = Console.ReadLine();
            a = int.Parse(s);
            s = Console.ReadLine();
            b = Convert.ToInt32(s);
            c = a + b;
            Console.WriteLine("c = {0}",c);
      }
  }
}
```

Файл проекта

Строится автоматически, имеет название: Имя_проекта.csproj.

Формируется в xml. Например:

```
<Project Sdk="Microsoft.NET.Sdk">
   <PropertyGroup>
      <OutputType>Exe</OutputType>
      <TargetFramework>net6.0</TargetFramework>
      <ImplicitUsings>enable</ImplicitUsings>
      <Nullable>enable</Nullable>
   </PropertyGroup>
</Project>
```

Тип проекта

Версия

Фрагмент описания проекта в Astra Linux

```
<?xml version="1.0" encoding="utf-8"?>
<Project DefaultTargets="Build" ToolsVersion="4.0"
                                        xmlns="http://schemas.microsoft.com/developer/msbuild/2003">
  <PropertyGroup>
      <Configuration Condition=" '$(Configuration)' == '' ">Debug</Configuration>
      <Platform Condition=" '$(Platform)' == '' ">x86</Platform>
      <ProjectGuid>{A869356D-1F1D-4250-B7A2-3843661C8F7B}</ProjectGuid>
      <OutputType>Exe</OutputType>
      <RootNamespace>Ex01_01</RootNamespace>
      <AssemblyName>Ex01_01</AssemblyName>
      <TargetFrameworkVersion>v4.5</TargetFrameworkVersion>
      <ProductVersion>8.0.30703</ProductVersion>
      <SchemaVersion>2.0</SchemaVersion>
  </PropertyGroup>
  <PropertyGroup Condition=" '$(Configuration)|$(Platform)' == 'Debug|x86' ">
      <DebugSymbols>true</DebugSymbols>
      <DebugType>full</DebugType>
      <Optimize>false</Optimize>
      <OutputPath>bin\Debug</OutputPath>
          ...
  </PropertyGroup>
  <ItemGroup>
     <Reference Include="System" /> ...
```